

1. You are given an $N \times N$ chessboard. The objective is to place N queens on the board such that no two queens threaten each other. That is, no two queens should share the same row, column, or diagonal.

Pseudo Code

1. N Queens backtracking Pseudocode

```
FUNCTION solveNQueens(N):  
    board = an array of size N, initialized to -1  
    solutions = empty list to store valid solutions  
    CALL solve(0, N, board, solutions)  
    RETURN SOLUTIONS
```

```
FUNCTION solve(row, N, board, solutions):  
    IF row == N THEN  
        ADD a copy of board to solutions  
        RETURN
```

```
    FOR col FROM 0 to N-1 DO:
```

```
        IF isSafe(board, row, col, N) THEN  
            board[row] = col
```

```
            CALL solve(row+1, N, board, solutions)
```

```
            board[row] = -1
```

```
FUNCTION isSafe(board, row, col, N):
```

```
    FOR i FROM 0 TO row-1 DO:
```

```
        IF board[i] = col THEN  
            RETURN False
```

```
    FOR i FROM 0 TO row-1 DO:
```

```
        IF board[i] - i = col - row THEN  
            RETURN False
```

```
    FOR i FROM 0 TO row-1 DO:
```

```
        IF board[i] + i = col + row THEN  
            RETURN False
```

```
    RETURN TRUE
```

Algorithm

Algorithm

Step 1: Define a function `solveNQueens(N)` that initializes the chessboard and calls the recursive function `solve`.

Step 2: Define a recursive function `solve(row)`:

(2a) If `row == N`, add the current board configuration to the solution list.

(2b). For each column `col` in current row:

→ If placing a queen at `board[row] = col` is safe:

(i) Place the queen at `board[row] = col`

(ii) Recursively call `solve(row+1)` to place a queen in next row.

(iii) If recursive call finishes, backtrack by removing the queen.

Step 3: Define a function `isSafe(row, col)` that checks if placing a queen at position `(row, col)` is safe:

~~Step 4: Check~~

Step 4: Check all previously placed queens to ensure:

(a) No queen is in the same column.

(b) No queen is in the same left diagonal.

(c) No queen is in the same right diagonal.

Step 5: Return the solutions list containing all valid configurations.

Code

```
1 def is_safe(board, row, col, N):
2     # Check if there's a queen in the same column
3     for i in range(row):
4         if board[i] == col:
5             return False
6
7     # Check if there's a queen in the same diagonal (left-to-right diagonal)
8     for i in range(row):
9         if board[i] - i == col - row:
10            return False
11
12    # Check if there's a queen in the same diagonal (right-to-left diagonal)
13    for i in range(row):
14        if board[i] + i == col + row:
15            return False
16
17    return True
18
19 def solve_n_queens(N):
20     def solve(board, row, N):
21         if row == N:
22             # All queens are placed successfully, store solution
23             solutions.append(board[:])
24             return
25
26         # Try placing a queen in each column of the current row
27         for col in range(N):
28             if is_safe(board, row, col, N):
29                 board[row] = col # Place queen
30                 solve(board, row + 1, N) # Try placing queen in next row
31                 board[row] = -1 # Backtrack if no solution is found in further rows
32
33     solutions = []
34     board = [-1] * N # Initialize board where board[i] represents the column where the queen is placed in the i-th row
35     solve(board, 0, N)
36
37     return solutions
38
39 # Example usage:
40 N = 4 # 8-queens problem
41 solutions = solve_n_queens(N)
42 print(f"Number of solutions for {N}-Queens: {len(solutions)}")
43 for solution in solutions:
44     print(solution)
```

Output

```
In [3]: runfile('C:/Users/Niranjan/nqueens_daa.py',
wdir='C:/Users/Niranjan')
Number of solutions for 3-Queens: 0

In [4]: runfile('C:/Users/Niranjan/nqueens_daa.py',
wdir='C:/Users/Niranjan')
Number of solutions for 4-Queens: 2
[1, 3, 0, 2]
[2, 0, 3, 1]
```

Implementation

1. Function Definitions:

is_safe(board, row, col, N):

- Checks if it is safe or not to place a queen at row 'row', column 'col'.
- Returns True if safe, False otherwise.

solve_n_queens(N):

- Solves the N-Queens problem for a board of size N.
- Returns a list of all the possible solutions.

2. is_safe(board, row, col, N) Implementation :

- Check if there is a queen in the same column. Run a loop from row = 0 to row - 1.
- If there is a queen in the same column, i.e board[i] == col then return False
- See if there is a queen in the same diagonal : for rows from 0 to row-1
- If there's a queen in the same diagonal i.e., board[i] - i == col - row then return False
- See if there is a queen in the same diagonal : for rows from 0 to row - 1.
- If a queen is found in the same diagonal, i.e., board[i] + i == col + row, return False. If no conflicts were found, return True.

3. solve_n_queens(N) Implementation:

- Solution: Create a recursive function solve(board, row, N): If the row is equal to N, this means all queens are placed successfully. Append a copy of the board to solutions list. Return for col in range(0,N): if it's safe to place a queen at row, col using is_safe :
- Places a queen at row and col i.e., board[row] = col.
- Recursive call solve(board, row+1, N) to try to place the next queen.
- Backtrack. Remove the queen. i.e., board[row] = -1
- Initialise an empty list solutions to store solutions.
- Create a board of size N initialized with -1 indicating no queen is placed.
- Call solve(board, 0, N) to start the recursive solving process
- Return the solutions list.

Time Complexity

Time complexity: $O(N!)$, since the solution explores all permutations of placing N queens on the board.

This algorithm recursively places queens and backtracks when no valid solution is found, exploring all possible configurations of queen placements on the board. Yes, it is an optimal solution for Nqueens problem.

2. Given a set of integers and a target sum, determine if there is a subset of the given integers that adds up to the target sum. The solution must use backtracking to explore all possible subsets.

Pseudo Code

2. Sum of Subsets problem

Pseudo-code

```
FUNCTION subsetSum(arr, n, target):  
    IF target == 0 THEN  
        RETURN True  
    IF n == 0 AND target != 0 THEN  
        RETURN False  
    IF subsetSum(arr, n-1, target) THEN  
        RETURN True  
    IF arr[n-1] <= target THEN  
        IF subsetSum(arr, n-1, target - arr[n-1])  
            THEN  
            RETURN True  
    RETURN False
```

Algorithm

Algorithm

Step 1: Base Case 1, if target becomes 0, this means a subset that adds up to the target sum has been found. Return True.

Step 2: Base case 2, if no elements are left to process ($n == 0$) and target is not zero, return False.

Step 3: Recursive step.

option 1: Exclude the current element from the subset and recursively check for the remaining elements with the same target.

option 2: If the current element is less than or equal to target, include it in subset. Subtract its value from the target and recursively check for the remaining elements.

Step 4: Return the result.

Step 5: If either excluding or including the current element results in a subset that adds up to target, return True. Else, return False.

Code

```
1 def subset_sum(arr, n, target):
2     # Base Case 1: If the target becomes 0, we have found a valid subset
3     if target == 0:
4         return True
5
6     # Base Case 2: No elements left and the target is not achieved
7     if n == 0 and target != 0:
8         return False
9
10    # Recursive Case 1: Exclude the current element and move to the next
11    if subset_sum(arr, n - 1, target):
12        return True
13
14    # Recursive Case 2: Include the current element if it is <= target
15    if arr[n - 1] <= target:
16        if subset_sum(arr, n - 1, target - arr[n - 1]):
17            return True
18
19    # If neither inclusion nor exclusion leads to a solution, return False
20    return False
21
22    # Example usage
23    arr = [3, 34, 4, 12, 5, 2]
24    target = 9
25    n = len(arr)
26
27    if subset_sum(arr, n, target):
28        print("Found a subset with the given target sum.")
29    else:
30        print("No subset found with the given target sum.")
```

Output

```
In [5]: runfile('C:/Users/Niranjan/sum_of_subset.py',
wdir='C:/Users/Niranjan')
Found a subset with the given target sum.
```

Implementation

- Function Call: The following call is made to the function subset_sum: playwright
- arr: This is the list of integers given to the function: [3, 34, 4, 12, 5, 2]
- n : This is the size of the list passed into the function: 6
- target: This is target sum for which we are searching for a subset that sums up to 9.

Recursive Calls:

- The function would recursively go through different subsets of the list in an attempt to find one whose sum equates to 9.
- First call: subset_sum([3, 34, 4, 12, 5, 2], 6, 9)
 - Since target is not 0 and there are elements remaining, the function goes to the recursive cases,
 - Recursive Case 1: Excluding the last element : subset_sum([3, 34, 4, 12, 5], 5, 9)
 - Recursive Case 2: Including the last element : subset_sum([3, 34, 4, 12, 5], 5, 0)

Base Case:

- Second recursive call: subset_sum([3, 34, 4, 12, 5], 5, 0)
 - The base case is satisfied because the target is 0, and there are remaining elements.
 - The function will return True because a subset-an empty subset-can sum up to 0.

Backtracking:

- True backtracks the propagation upward through recursive calls.
- The function returns True to the initial call, which will mean a subset with a given target sum does exist.

Time Complexity

The time complexity of this backtracking solution is $O(2^n)$, where n is the number of elements in the array. This is because we are considering all subsets of the array, which is $2n2^n$ in the worst case.